Scaling Laws for Language Models

From Power Laws to Compute-Optimal Training

CS115B – Spring 2026

James Pustejovsky

April 14, 2026

Today's Question

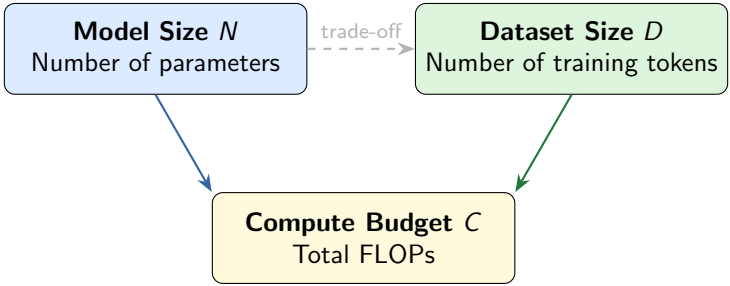
If someone gave you $100 million to train
the best possible language model,
how would you spend it?

This is not hypothetical – companies face exactly this decision. The answer changed dramatically between 2020 and 2022, and it's still evolving.

Today: The science behind that decision.
Next time: How modern architectures (Mixture of Experts) put that science into practice.

Three Knobs to Turn

Every training run is defined by three quantities:



$$C \approx 6ND$$

The $6ND$ rule: forward pass $\approx 2N$ FLOPs/token, backward $\approx 4N$, times D tokens.

The central question: Given a fixed budget C, how do you split between a bigger model (N) and more data (D)?

Why Cross-Entropy Loss?

Scaling laws predict **cross-entropy loss**, not task accuracy. Why?

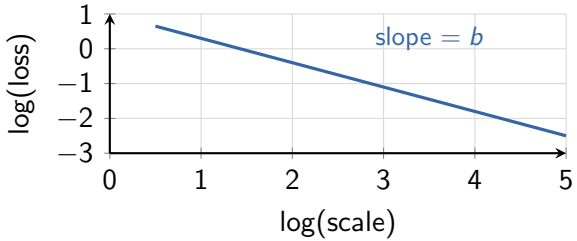
$$L = -\frac{1}{D}\sum_{i=1}^{D} \log P_\theta(x_i \mid x_{<i})$$

- **Smooth and continuous** – you can fit curves to it
- **Well-defined** across model sizes and architectures
- **Connected to perplexity:** $\text{PPL} = e^L$
- **Lower bound:** Entropy of natural text ≈ 1.69 nats – no model can do better

Task accuracy is noisy, task-dependent, and can show discontinuities. Loss is the clean signal that makes scaling laws possible.

What Is a Power Law?

A **power law**: $y = a \cdot x^b$. In log-log space, this is a straight line.



NLP has known about power laws for a century: Zipf's law (word frequencies), Heaps' law (vocabulary growth).

The discovery: LM loss also follows power laws in N, D, and C – over *seven orders of magnitude*. This means you can predict a 100B model's loss from experiments at 100M.

Kaplan et al. (2020): The First Scaling Laws

Paper

Kaplan, McCandlish, Henighan, Brown, Chess, Child, Gray, Radford, Wu, Amodei.
Scaling Laws for Neural Language Models. OpenAI, January 2020.

Four key findings:

1. Loss follows power laws in N, D, and C – remarkably predictable
2. **Architecture details don't matter much:** Width vs. depth, number of heads, FFN ratio – all secondary to total parameter count
3. **Larger models are more sample-efficient:** A bigger model reaches the same loss with fewer tokens
4. **Scale N faster than D**: For $10\times$ more compute, make the model $5.5\times$ bigger but use only $1.8\times$ more data

Finding 4 directly shaped GPT-3's design. **But it would turn out to be wrong.**

The Kaplan Recipe

Kaplan recommended: $N_{\text{opt}} \propto C^{0.73}$, $D_{\text{opt}} \propto C^{0.27}$.

In plain English: When you get more compute, mostly make the model bigger. Don't worry too much about more data.

The industry followed this advice:

Model	Parameters	Training Tokens
GPT-3 (OpenAI, 2020)	175B	300B
Gopher (DeepMind, 2021)	280B	300B
Megatron-Turing NLG (2022)	530B	270B

Notice: nearly the same amount of data every time, despite 3× bigger models.

Question: Should a 530B model really see the same data as a 175B model?

What Kaplan Got Right (and Wrong)

Right – and still standing:

- Loss follows power laws (confirmed by everyone since)
- Architecture details matter less than scale
- Larger models are more sample-efficient

Wrong – due to a methodological issue:

- All models were trained with the *same* long cosine LR schedule
- When estimating loss for shorter training runs, they read off *intermediate* values
- At those intermediate points, the learning rate hadn't decayed yet – loss was artificially high
- This made short training look worse than it actually is
- **Bias:** Favor bigger models trained for fewer steps

If you match the LR schedule to each training horizon, the picture changes dramatically.

Hoffmann et al. (2022): The Chinchilla Correction

Paper

Hoffmann, Borgeaud, Mensch, et al.
Training Compute-Optimal Large Language Models. DeepMind, March 2022.

Central claim: Existing LLMs are **significantly undertrained**. For the same compute, a *smaller* model on *more data* wins.

Evidence:

- Trained 400+ models (70M to 16B params, 5B to 500B tokens)
- Three independent analysis methods, all converging on the same answer
- Validated by training Chinchilla: 70B params, 1.4T tokens

Chinchilla used the *same compute* as Gopher (280B, 300B tokens) but outperformed it on nearly every benchmark. 4× smaller, 4× more data.

Three Approaches, One Answer

Hoffmann et al. used three independent methods to find the optimal (N, D) for a given C:

Method	a **where** $N_{\text{opt}} \propto C^a$	b **where** $D_{\text{opt}} \propto C^b$
1. Training curve envelope	0.50	0.50
2. IsoFLOP profiles	0.49	0.51
3. Parametric loss function	0.46	0.54
Kaplan et al. (2020)	0.73	0.27

All three say the same thing: scale N and D equally. $a \approx b \approx 0.5$.

For $10\times$ more compute: Kaplan says $5.5\times$ model / $1.8\times$ data. Chinchilla says $\sim 3.2\times$ each.

The Parametric Loss Function

Hoffmann's third approach gives us an intuitive decomposition:

$$\hat{L}(N, D) = \underbrace{E}_{\substack{\text{entropy of} \\ \text{natural text}}} + \underbrace{\frac{A}{N^{\alpha}}}_{\substack{\text{model isn't} \\ \text{big enough}}} + \underbrace{\frac{B}{D^{\beta}}}_{\substack{\text{haven't seen} \\ \text{enough data}}}$$

Three sources of loss:

1. $E \approx 1.69$: The floor. Even a perfect model can't predict language perfectly.
2. A/N^{α}: The model capacity penalty. Shrinks as you add parameters.
3. B/D^{β}: The data penalty. Shrinks as you train on more tokens.

To minimize $\hat{L}$ under the constraint $C = 6ND$, you need to *balance* the two penalty terms. If you spend all your compute on a huge model with little data, B/D^{β} dominates. If you train a tiny model on mountains of data, A/N^{α} dominates.

The optimum: balance them. This is why the exponents come out near 0.5.

The 20× Rule

From Hoffmann's analysis, a simple rule of thumb:

Chinchilla Rule of Thumb

A compute-optimal model should train on roughly **20 tokens per parameter**.

Parameters	Optimal Tokens	Ratio
1B	20B	20×
10B	205B	∼20×
70B	1.5T	∼21×
175B	3.7T	∼21×

GPT-3 (175B) trained on 300B tokens – a ratio of 1.7×. By this analysis, it needed 12× more data. It was severely undertrained.

Chinchilla vs. Gopher: The Experiment

	Gopher	Chinchilla
Parameters	280B	70B
Training tokens	300B	1.4T
Training FLOPs	5.76×10^{23}	5.76×10^{23}

Same compute budget. $4\times$ fewer parameters, $\sim 5\times$ more data.

Result: Chinchilla outperforms Gopher on:

- Every subset of The Pile (language modeling)
- MMLU: 67.6% vs. 60.0% (knowledge)
- RACE-h: 82.3% vs. 71.6% (reading comprehension)
- HellaSwag, Winogrande, BoolQ, LAMBADA ...

And being $4\times$ smaller, Chinchilla is $\sim 4\times$ cheaper to serve at inference.

Did the Industry Listen?

Model	Params	Tokens	Tokens/Params
Pre-Chinchilla (Kaplan recipe)			
GPT-3 (2020)	175B	300B	1.7×
Gopher (2021)	280B	300B	1.1×
Chinchilla-era			
Chinchilla (2022)	70B	1.4T	20×
Llama 1 (2023)	65B	1.4T	22×
Post-Chinchilla (over-training)			
Llama 2 (2023)	70B	2.0T	29×
Llama 3 (2024)	70B	15T	214×
Mistral 7B (2023)	7B	>1T	>140×

Notice: recent models train on *far more* than 20× their parameters. Why?

Over-Training: When Inference Cost Matters More

Chinchilla-optimal minimizes loss per *training* FLOP.
But in production, *inference* cost dominates. Training is paid once; inference is paid every query.

The inference-aware trade-off:

- A 70B model costs $\sim 10\times$ more per query than a 7B model
- If you serve billions of queries, inference $\gg$ training cost
- **Solution:** Train a *smaller* model for *much longer* than Chinchilla-optimal

Example: Llama 3 8B trained on 15T tokens ($1875\times$ params).
Chinchilla says an 8B model needs $\sim$160B tokens. Llama 3 uses $\sim 94\times$ more than that.

The extra training compute is a one-time cost that buys a smaller, cheaper model forever.

The Data Wall

If compute-optimal training needs $\sim 20\times$ tokens per parameter:

Model Size	Optimal Tokens	Challenge
1B	20B	Easy
70B	1.4T	Achievable
1T	20T	$\approx$ total high-quality web text
10T	200T	Exceeds all available text

Rough estimates of available high-quality text: $\sim$10–20T tokens total (web, books, code, academic papers combined).

Implication: We may run out of data before we run out of compute. This is one of the pressures driving interest in synthetic data, multi-modal training, and architectures that are more data-efficient.

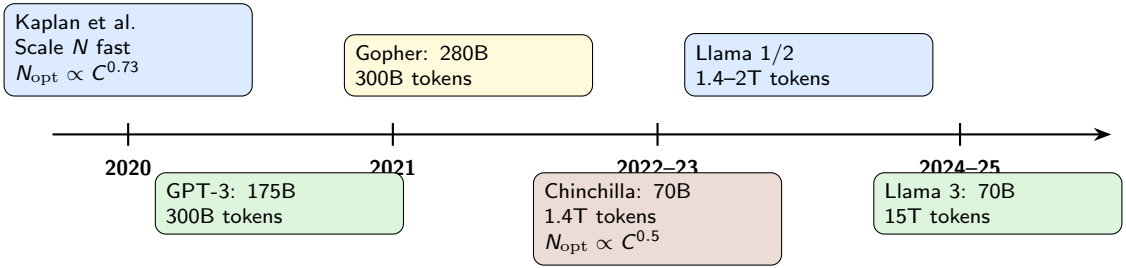
A Different Kind of Scaling

Scaling laws tell us about making models bigger. But there are other ways to scale:

- **Inference-time compute:** Chain-of-thought, best-of-N sampling, reasoning models (o1, R1) – spend more compute *per question*
- **Data quality:** A curated token may be worth 5–10$\times$ a random web token. Can we define "effective tokens"?
- **Sparse activation:** Have many parameters, but only use a subset per token. This is **Mixture of Experts** – our next lecture.

MoE is in some sense the natural response to the scaling dilemma: if more parameters help but more compute hurts, *decouple* the two.

Timeline



The arc: Scale parameters (Kaplan) → Scale data equally (Chinchilla) → Over-train small models for inference (Llama 3) → **Use sparse models to get both** (MoE – next time).

Key Takeaways

1. **Loss follows power laws** in N, D, and C – predictable over seven orders of magnitude.
2. **Kaplan (2020)** found the laws and recommended scaling N faster than D. A methodological issue (fixed LR schedule) led to this bias.
3. **Chinchilla (2022)** corrected the allocation: scale N and D equally ($\sim C^{0.5}$ each). Most 2020–22 LLMs were undertrained.
4. **The 20× rule:** Train on ∼20 tokens per parameter for compute-optimality.
5. **In practice,** inference cost pushes toward smaller models trained much longer (over-training).
6. **The data wall** looms: at trillion-parameter scale, we may not have enough text.

Next Time

Mixture of Experts: architectures that decouple capacity from cost.
Read: NVIDIA Nemotron 3 Super Technical Report (§2.1).

References

Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.

Brown, T.B., et al. (2020). Language models are few-shot learners. *NeurIPS 2020*.

Rae, J., et al. (2021). Scaling language models: Methods, analysis & insights from training Gopher. *arXiv:2112.11446*.

Touvron, H., et al. (2023). Llama: Open and efficient foundation language models. *arXiv:2302.13971*.

Touvron, H., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.

Sutton, R. (2019). The bitter lesson. *Blog post*.